4.5 Composer and PSR-4

PSR means PHP Standard Recommendation. PHP-FIG's PSR-4 specifies how namespace prefixes map to base directories and how the rest of a class name maps to a file. It is an interoperability convention that Composer supports, not a PHP keyword.

Configure the prefix

Merge this top-level member into `composer.json`, preserving `require`, `scripts`, and metadata:

```
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    }
}
```

The JSON spelling `"App\\"` represents the prefix `App\`: JSON escapes a backslash with another backslash. The trailing namespace separator matters. The directory is relative to `composer.json`.

Class name	Remove prefix	Resulting file
`App\Tutorials\Class4\User`	`Tutorials\Class4\User`	`src/Tutorials/Class4/User.php`
`App\Tutorials\Class4\Admin\User`	`Tutorials\Class4\Admin\User`	`src/Tutorials/Class4/Admin/User.php`
`App\Tutorials\Class4\Services\WelcomeMessage`	`Tutorials\Class4\Services\WelcomeMessage`	`src/Tutorials/Class4/Services/WelcomeMessage.ph`

The last row predicts a file we have not created. Match directory and filename case exactly. PSR-4 requires case consistency even if a local filesystem appears forgiving. One named class per matching file is the convention used here.

Generate and activate the loader

```
composer validate
composer dump-autoload
```

This generates loading code under `vendor/`. Plain `dump-autoload` does not enable optimized classmap generation by default; optimization is a separate option. Do not edit generated vendor files.

Create `public/tutorials/4-5-composer-and-psr-4.php`:

```
<?php
declare(strict_types=1);

use App\Tutorials\Class4\Admin\User as AdminUser;
use App\Tutorials\Class4\User;

// Composer's loader registers a callback for PSR-4 and other autoloading rules
// Commonly called the Composer bootstrap file, it is generated by Composer and should not be edited
require_once __DIR__ . '/../../vendor/autoload.php';

echo (new User('Avery'))->getName(), '<br>', PHP_EOL;
echo (new AdminUser('Morgan'))->getName(), '<br>', PHP_EOL;
```

Run `php public/tutorials/4-5-composer-and-psr-4.php` and expect the same two lines. The custom callback and individual class requires are now gone. The one bootstrap require registers Composer's loader; classes are loaded as needed afterward.

When changing the autoload configuration, rerun `composer dump-autoload`. A new correctly placed class beneath the existing prefix normally needs no regeneration with the default PSR-4 loader. Authoritative classmaps have different regeneration requirements and are outside this lesson.

Diagnose a missing class

Temporarily change only `namespace App\Tutorials\Class4\Admin;` to `namespace App\Tutorials\Class4\Admins;` in the admin file. The loader can locate the file, but it does not declare the requested class, so construction fails. Restore the namespace.

Check failures in this order: bootstrap path, configured prefix, generated loader, exact namespace, directory case, class name, filename. A namespace or case mismatch is not fixed by adding random requires. A missing vendor directory requires regeneration or installation.

References: PSR-4 specification and Composer autoload schema.